Parallelism In Chess Search Trees

github.com/Sayeem2004/CSLChess

Zubin Bhaumik
zb57@cornell.edu

Mohammad Khan
msk295@cornell.edu

May 14, 2026

1. Project Hypothesis and Goals

This project investigates the parallel efficiency (PE) and competitive game performance of two common search tree algorithms: Alpha-Beta Pruning (ABP) and Monte Carlo Tree Search (MCTS). These algorithms have an inherent difference in search strategy: ABP does an efficient complete search to a certain depth, while MCTS does a stochastic search but delves much deeper and tries to focus on promising moves. Our central hypothesis was that MCTS would exhibit superior scaling and parallel efficiency because its simulation-based nature is more embarrassingly parallel than the synchronized pruning of ABP. On the other hand, we expected ABP to dominate in competitive game performance because of its complete nature, whereas MCTS's stochastic approach might overlook critical moves in games with relatively low branching factors like chess (MCTS is usually more suited for larger branching factors like in Go).

This investigation is significant because chess search trees are highly irregular workloads. Developing effective parallelization strategies for such trees could provide valuable insights that can be applied to other complex non-uniform search problems like in combinatorial optimization of TSP, SAT, or protein folding. One conclusion we found is that there is a stark tradeoff between improving search efficiency and improving parallel efficiency. For example, as we improved our ABP implementation to prune significantly more suboptimal nodes through prioritizing capture and killer moves our parallel efficiency decreased because the workload shrunk significantly and became more irregular making it hard to justify the overheads of parallelism. In the case of MCTS, we discovered that the almost embarrassingly parallel Root Parallelism strategy was simple to scale but performed a lot of duplicated work. When we transitioned to a more efficient Tree Parallelism strategy we reduced this duplication, but introduced more synchronization overhead limiting parallel performance. In the case of chess, these efficiency improvements were too significant to ignore, but this may not be the case in other search problems.

In terms of competitive game performance, our results confirmed that complete search (ABP) remains significantly better than stochastic simulations (MCTS) for chess performance. Furthermore, consistent with our expectations even a single-threaded version of Stockfish dominated both our algorithms in competitive game performance. This is due to the many varied simpler searches that Stockfish performs on top of Alpha Beta pruning (allowing it to explore far more useful nodes than our uniform search). From here we can conclude that uniform searches are often less effective than combining multiple non-uniform searches. However, again this comes at the cost of parallel performance as different searches increase the irregularity of each threads workload.

2. Project Context and Contributions

2.1. Project Context

The motivation for this project stems from the desire to apply high-performance parallel techniques on an algorithmically complex problem. We were introduced to search algorithms like Alpha-Beta Pruning (ABP) and Monte Carlo Tree Search (MCTS) in introductory AI courses (such as CS 4700), and we wanted to apply the parallelization techniques we learned throughout the semester to these problems, while also having the fun goal of creating a good chess engine to try and beat Stockfish.

Originally, we intended to also explore these search patterns on the Cerebras WSE-2 architecture to leverage its low-latency on-chip communication. However, due to the technical complexity of implementing a bitboard chess system from scratch and the strict memory limits of individual processing elements, we pivoted to only using CPU specific architectures like OpenMP and MPI that could use prebuilt bitboard libraries.

This project serves as a case study in irregular workload distribution and search trees, relevant to domains like combinatorial optimization, search-based planning, and protein folding. This is an independent project and is **not** a sub-project of a larger research project or being submitted to any other classes. Others could build on top of our work if they want to further explore the complexities of creating a good chess engine and the pain of challenging Stockfish.

2.2. Team Contributions

The workload for this project was divided based on the two search algorithms investigated, with joint efforts focused on benchmarking and presentation. Mohammad Khan implemented and optimized the ABP algorithm with OpenMP and MPI, while also writing a majority of the parallel efficiency benchmarking suite. Zubin Bhaumik implemented and optimized the MCTS algorithm with OpenMP and MPI, while also writing a majority of the Stockfish competitive analysis code. We equally split the tasks of data visualization, poster presentation design, and the final report.

3. Prior Work

We used 4 relevant prior works/resources to guide our parallel implementation of ABP/MCTS and provide useful benchmarks and prebuilt functions that were not worth re-implementing.

Singhal and Sridevi [4] parallelized Alpha-Beta Pruning (ABP) on CPU and GPU architectures, achieving speedups of $3.23\times$ and $7.26\times$, respectively. We used their work to guide our initial Alpha-Beta implementation, however we soon realized that their implementation was limited due to its simplistic Minimax structure with separate logic for each player. We extended their approach by implementing various techniques from the Chess Programming Wiki [6] like Negamax and transposition tables to improve pruning efficiency. Additionally, while the original study focused on OpenMP in the CPU case, we also tested out multi-CPU parallelism through MPI.

Kurzer et al. [2] defined the fundamental strategies for parallelizing Monte Carlo Tree Search (MCTS), specifically Root, Leaf, and Tree parallelism of which we used Root and Tree parallelism. Their research focused on continuous domains within multi-agent reinforcement learning, whereas we adapted these architectural patterns to the discrete domain of chess. We used their work to structure our initial MCTS implementation, but also introduced low-level optimizations like lock-free atomic updates and cache-line alignment padding to reduce synchronization bottlenecks and hopefully increase parallel efficiency.

To allow for reasonable competitive game performance and parallel efficiency benchmarks, we utilized the Stockfish engine [5] and the Chess Programming Wiki [6]. We specifically integrated the board evaluation function from Stockfish 11 (2020) for two reasons. First, we thought its hand-crafted logic (as opposed to a neural network) would have worked better in CSL if we got to it, but even without it we think it would help us slightly reduce the computational bottleneck of evaluating leading to more useful parallelization results. Second, even if we created a similarly strong ABP or MCTS search algorithm to Stockfish if we don't have a good evaluator our competitive game performance wouldn't be comparable. The Chess Programming Wiki provided the theoretical understanding for our advanced pruning and search order improvements. Unlike these resources, which apply many different search algorithms at the same time to maximize ELO, we limited our implementation to using just our main ABP/MCTS algorithms to balance algorithmic complexity/efficiency and consistent benchmarking results (other search algorithms would distract from our main focus of ABP and MCTS).

4. Empirical Methodology and Results

4.1. Experiment Design

As described above, we implemented ABP and MCTS using OpenMP and MPI in C++. The input to these functions is a board position and some budget. The two algorithms would explore the search trees and continually update the best move until the budget runs out. We defined three types of budgets: depth-bound, time-bound, and cycle-bound. Depth-bound limits the search to a certain depth, but only really makes sense for ABP because MCTS needs to simulate until game completion for competitive performance (can still be used for parallel efficiency). Under time-bound constraints ABP performs iterative deepening, while MCTS continually performs batches of simulation until the budget is reached. Finally, cycle-bound is a bit unconventional and was originally designed to provide a fair comparison with CSL simulators. Even so we thought that since our algorithms are more simplistic than Stockfish, we could get an advantage under cycle constraints. To implement cycle bounds in ABP/MCTS we used the `RDTSC x86` instruction, while for Stockfish we used `perf stat -e` to record the cycles across various Stockfish runs, estimate ~ 1800 cycles per node, and limit based on node count.

For benchmarking performance, we had a dataset of 100 early game, 100 mid game, and 100 late game positions described by FEN strings obtained from the Lichess Puzzles Dataset [3]. We varied across the game stages in order to experience different branching factors and situations. There is a slight bias as chess puzzles are usually in a situation where capturing pieces are more likely, but we didn't think this significantly affected our benchmarking especially since we varied across game states. We benchmarked on Perlmutter which has AMD EPYC CPUs with up to 256 OpenMP threads and up to 2 MPI ranks. Throughout our optimization process, we kept optimizations that improved the time to complete a fixed depth task (usually 5) and/or improve the number of nodes/simulations completed within a fixed time limit (usually 500 ms). We also kept track at how parallel efficiency was affected at various optimization checkpoints.

For competitive game performance, we played against Stockfish 11 which can range in ELO from 1350 (near average Lichess ELO) up to 2850 (Magnus Carlsen ELO, but not superhuman). To play games against Stockfish we used a Python interface to our various C++ functions (depending on an open-source bitboard library [1]) and limited through all 3 bounds for Alpha-Beta and the 2 latter bounds for MCTS (no depth-bound). For each bound we had a small, medium, and large budget differing by about a magnitude. A win counts as 1 point, a draw 0.5, and a loss 0. We played 100 games and counted our algorithm better than Stockfish if we passed the break-even point of 50.

4.2. Monte Carlo Tree Search

4.2.1 MCTS Background

Monte-Carlo Tree Search (MCTS) works by iteratively building a probabilistic search tree to determine moves. Each iteration consists of four distinct phases:

- **Selection**: The search begins at the root node. The algorithm recursively selects optimal children until it reaches a leaf node (a node with unexpanded legal moves). The optimality of a node is calculated using the UCT (Upper Confidence Bound applied to Trees):

$$\text{UCT} = \left(1.0 - \frac{w}{v}\right) + c\sqrt{\frac{\ln(V)}{v}}$$

  where w is the win statistic, v is the node's total visits, V is the parent node's visits, and c is an exploration constant (tuned to 1.414).
- **Expansion**: An untried legal move is expanded, instantiating a new child node.
- **Simulation**: From the child node, random moves are played until a terminal state or depth limit is reached. The board is then resolved into a score of $\{0.5, 1, -1\}$ for a stalemate, victory, or loss; if legal moves are still available, an evaluation function is used instead.
- **Backpropagation**: The score is propagated to the root, and each node has its total visits and win score updated.

MCTS then selects a move based on a heuristic; in this case, the child node with the highest number of visits. MCTS can be parallelized three ways:

- **Root Parallelism (RP)**: Each thread copies the root node and performs MCTS independently, building up a **separate** tree. All of the trees' win/visit statistics are subsequently merged.
- **Leaf Parallelism (LP)**: When a child node is expanded, it is instantiated on all threads. Each thread then runs simulations from this node, and the statistics and expanded trees are merged.
- **Tree Parallelism (TP)**: A single search tree is shared among all threads during all phases of MCTS. Threads independently expand out child nodes, perform rollouts, and update scores.

In this project, root and tree parallelism are implemented and compared.

4.2.2 MCTS Implementation Effort

OpenMP was used to write a simple implementation of RP MCTS. The total simulation budget is divided across threads, each of which creates a copy of the root node to independently perform simulations/rollouts in parallel. Finally, the root statistics are merged on the main thread. Because each thread owns a separate tree, no locks or atomics are required, achieving excellent scalability. While Root Parallelism naturally scales without significant overhead by isolating each thread to its own subtree, it suffers from severe statistical inefficiency, as threads cannot share information about sub-optimal paths early in the search budget.

OpenMP was also used to write a TP implementation of MCTS. However, the use of a shared search tree across threads raised critical challenges that necessitated many algorithmic developments. To begin, during the **Selection** phase, multiple threads may reach the same high-UCT leaf nodes. This creates two challenges: (1) threads make decisions based on stale visit counts, leading to a similar statistical inefficiency to RP, and (2) race conditions during expansion, where multiple threads try to simultaneously populate an unexpanded child index within a shared node.

Issue (1) is solved via virtual losses. Upon descending into a node, a thread atomically increments a virtual loss counter using `virtual_loss.fetch_add(1, std::memory_order_relaxed)`. This then creates an effective node visit count of $v_{\text{effective}} = v + v_{\text{virtual}}$, which artificially decreases the node's apparent win rate. Subsequently, threads to divert to alternative paths, maximizing exploration across cores.

Issue (2) is solved by synchronizing node expansion using atomics. Specifically, each node contains atomics including `next_untried_idx` and an `is_stable` flag. Instead of locking the parent node to append a child, we perform lock-free expansion by pre-allocating the legal moves list. A thread claims an index via `fetch_add(1)` on `next_untried_idx`, and can instantiate and explore this child without interfering with other threads expanding adjacent indices. The node transitions to a stable state `is_stable = true` using `std::memory_order_release` when the last untried move has been assigned, permitting lock-free lookups for future traversals using `std::memory_order_acquire`.

Backpropagation is lock-free too; visits are incremented atomically via `node->visits.fetch_add(1)`. For wins, C++ atomics do not provide similar support for floating-point accumulation, so a compare-and-swap (CAS) pattern is used. The memory bottleneck is partially addressed by replacing the standard node/memory allocation via thread-local node pools. Standard heap allocation routines such as `new/malloc` rely on global OS-level locks. If thousands of simulations per second attempt to dynamically allocate individual MCTSNode instances concurrently, the memory allocator completely neutralizes scaling through lock contention.

The node pool is configured as a static `thread_local` variable, allocating memory out of large contiguous Chunk buffers (consisting of 2^{16} `MCTSNode` objects. This allows memory allocation to reduce to an atomic `c->used.fetch_add(1, std::memory_order_relaxed)`. The chunks are allocated to prevent false-sharing, via `static_cast<MCTSNode*>(::operator new(CHUNK_SIZE * sizeof(MCTSNode), std::align_val_t(64)))`. Within each `MCTSNode` object, an `alignas(64)` directive is used to allow each atomic statistic (wins, visits, virtual loss) to reside on a separate cache line. This prevents false sharing between threads accessing a node, and allows for concurrent updates without mutexes.

4.2.3 MCTS Results and Analysis

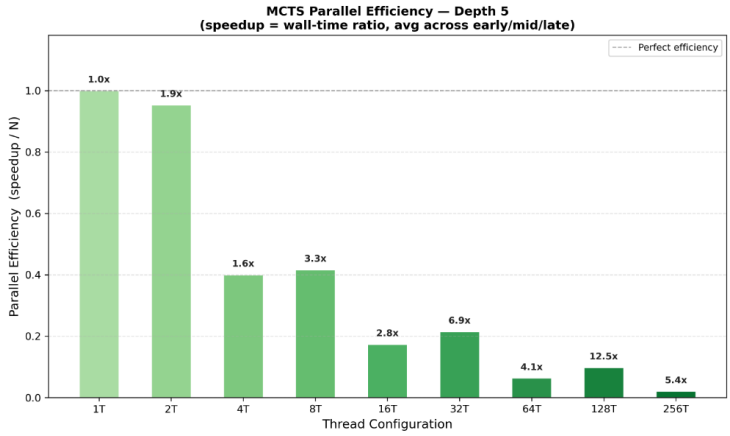

Figure 1: PE of depth-limited TP-MCTS across 1-256 threads.

The parallel efficiency of TP MCTS is shown in Figure 1, measured using the runtime required to complete 100K simulations to a rollout depth of 5. The algorithm demonstrates near-strong scaling from 1 to 8 threads, with speedup increasing almost proportionally to the available compute

resources. Beyond this point, however, scalability degrades significantly. Although performance continues to improve up to a maximum speedup of 12.5× at 128 threads, the gains are increasingly sublinear, and performance declines when scaling from 128 to 256 threads (both achieve PE $< 20\%$)

These results indicate that MCTS becomes bottlenecked by its limited ability to efficiently parallelize across large thread counts. The primary source of this limitation is the high synchronization overhead associated with the shared search tree. Each simulation requires frequent atomic updates, node expansions, and contention on shared tree structures, all of which serialize portions of execution. Additionally, the irregular access patterns inherent to tree-search algorithms reduce cache locality and increase cache coherence traffic and memory bandwidth pressure, further limiting scalability at high core counts.

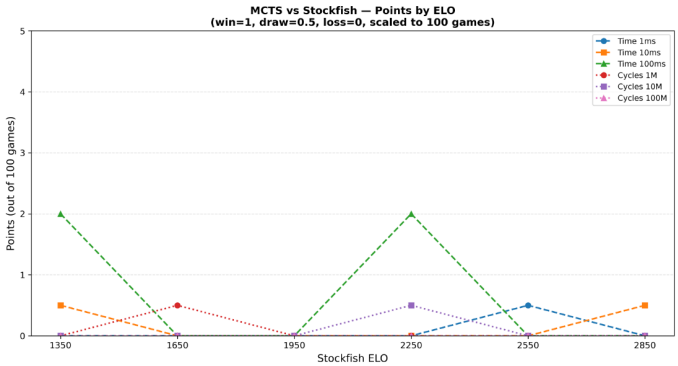

Figure 2

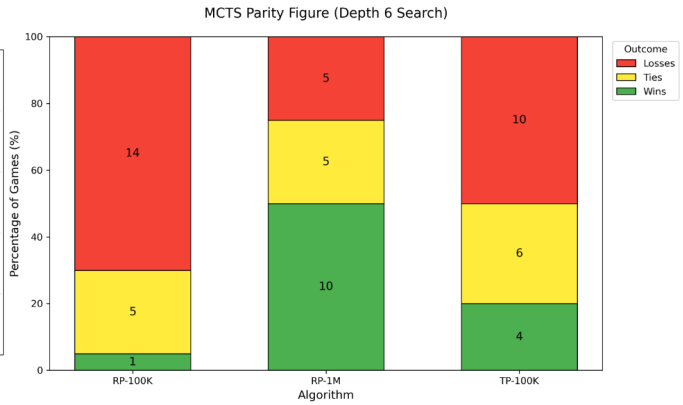

Figure 3

The competitive performance of the TP-MCTS algorithm is shown in Figure 2. Across most simulation budgets, TP-MCTS achieves very few wins against Stockfish at varying ELO levels. The strongest performance occurs under the 100ms time budget, suggesting that longer search windows allow the algorithm to perform more meaningful exploration of the search tree. In contrast, lower time budgets are heavily impacted by synchronization overhead, which limits the effective amount of useful search performed.

Figure 3 further highlights the inefficiency introduced by thread-parallel execution. RP-MCTS requires approximately 1M simulations to achieve parity with TP-MCTS at 100K simulations. Although the RP implementation experiences higher throughput (as TP is dominated by contention), duplication of work diminishes the quality of additional parallel search effort in RP.

4.3. Alpha-Beta Pruning

4.3.1 ABP Background

Alpha-Beta Pruning is an optimization of the Minimax search algorithm used to efficiently determine the optimal move in a two-player zero-sum game. In the setting of Minimax, one player aims to maximize an evaluation score, while the other player aims to minimize an evaluation score. The algorithm proceeds to a depth limit, evaluates all leaf board positions, and obtains the optimal move by moving upwards. At each parent node, the best move is chosen to be the move that leads to the child node with the worst score for the opponent. The value of this parent node is then set to be the value of the chosen child, and this process continues until we reach the root and have the best next move (given a fixed depth).

Alpha-Beta pruning optimizes the Minimax algorithm by maintaining two bounds: Alpha is the worst (lowest) score guaranteed for the maximizing player so far, and Beta is the worst (highest) score guaranteed for the minimizing player so far. Whenever the maximizing player is in a board position and finds a child node with a score higher than the guaranteed score for the minimizing player, it prunes the search tree. The minimizing player would not allow the game to reach this board position in the first place, so further children are irrelevant. This pruning process (and vice versa for the minimizing player) substantially decreases the effective branching factor of the search tree from over 30 to $\leq$10 provided that the child nodes are searched in a good order. This pruning also preserves the same optimal result as Minimax.

There are two main approaches to parallelize ABP, which we combine into a third hybrid choice:

- **Root Parallelism**: For the root node only, assign a thread to explore each child node. It is not effective to do this parallelism strategy recursively because of limited amount of OpenMP threads and the exponential number of nodes. This parallelism is limited by the irregular workloads of different child nodes (stragglers) and the limited amount of children at the root (usually $\sim$30).
- **Lazy SMP**: From our explanation above, ABP has different behavior based on the lowest depth chosen. A common engine strategy which we employ is to run a depth of 1 first, and then continue to run larger depths until your budget is exhausted. This process is called iterative deepening and Lazy SMP assigns a thread per depth searched in this process. This parallelism is limited by the feasible amount of depths you can calculate (usually around 8-10).
- **Hybrid**: On Perlmutter we have up to 256 nodes, which is beyond both the limits of Root Parallelism and Lazy SMP, but you can notice that $30 \times 8 \approx 256$. So a valid choice to get past both these limits is to pass in a thread group of size 32 to perform Root Parallelism in each of the 8 depth searches done by Lazy SMP. This ended up being the implementation with the best parallel efficiency/speedup so we used it for our benchmarking, but it is still limited due to irregular workloads and stragglers.

4.3.2 ABP Implementation Effort

The implementation of Alpha-Beta Pruning focused on maximizing both raw search speed through parallelism and the effectiveness of pruning through algorithm complexity to handle the exponential growth of the chess search space. However, these two optimizations counteracted each other through the development process. Below we go over some of the details of our optimization process.

First, we can discuss our parallelism based effort. Our ABP engine utilizes a Negamax framework to consolidate the Minimax logic into a single function by negating at every layer, so each player thinks they are maximizing. We used a lock free Transposition Table (TT) to allow different threads to cache their board evaluations together. The lock free nature means that some threads can miss bound updates and thus do unnecessary work, but it is worth it because we can avoid mutex contention. We applied a 32-byte TT alignment to minimize false sharing as opposed to a 64-byte alignment. Further testing revealed that a TT size of 4 million was optimal for the Perlmutter architecture, fitting efficiently within the L3 cache (128 MB / 256 MB).

Further parallelization effort centered on finding an optimal hybrid between Lazy SMP and Root Parallelism. Given the 256-thread capability of Perlmutter nodes, we tested various configurations (16x16, 4x64, and 8x32) determining that 8x32 was optimal. To further minimize synchronization overhead, we restricted global alpha checks to the root node, significantly reducing mutex contention when performing Lazy SMP. We also instantiate threads = root children if possible. If there are more than 32 root children, we use dynamic scheduling to handle the irregular workload.

To reduce the branching factor, we implemented a robust move-ordering system using cached TT moves first, then MVV-LVA (Most Valuable Victim - Least Valuable Aggressor, basically searching largest delta captures) second, and Killer Heuristics (basically children who cause pruning) third. This effort resulted in a massive reduction in search space: for example, at depth 4 in an early-game position, a naive search explores ~ 1.9 million nodes, ABP without move ordering explores $\sim 100k$ nodes, and ABP with move ordering explores 6245 nodes (a reduction of over 90% compared to before ordering). This increases absolute performance, but this optimization causes our 256 thread speedup to drop from 24.2× to 18.8× as pruning leads to irregular workloads and the low number of nodes can't offset the parallel overhead as well as before.

4.3.3 ABP Results and Analysis

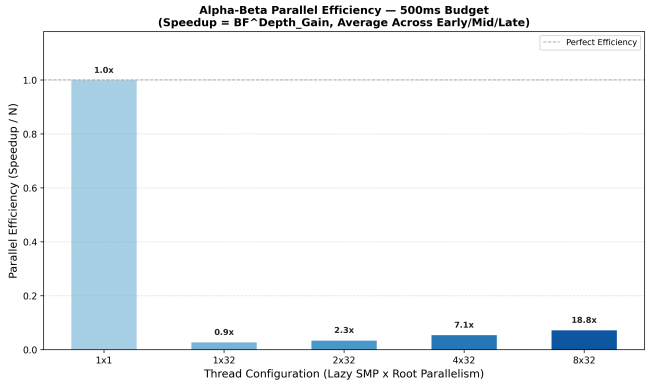

Figure 4

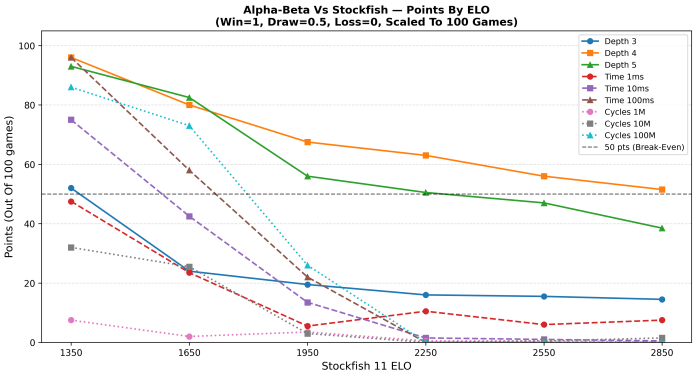

Figure 5

Figure 4 above shows the parallel efficiencies and speedups across multiple thread counts (1, 32, 64, 128, 256) when using the hybrid parallelism approach on a 500 ms time budget. We can notice that each doubling of threads actually leads to a more than 2× increase (other than 1T to 1x32T) and that is because nodes in the middle of searching actually drop all unfinished work when the time budget runs out. So a partially completed depth will not count towards the speedup leading to a bit of a misleading metric. However, we can notice that parallel efficiency is increasing as the amount thread increases. This is likely because Lazy SMP is more embarrassingly parallel and thus performing more of it brings the hybrid parallel efficiency up. This matches our earlier results from the poster presentation where pure Root Parallelism had 1.9% PE while Hybrid had a 9.5% PE with 8 groups meaning that the Lazy SMP had approximately a 66.4% PE between the two former situations. Despite this increase in PE, all PEs benchmarked are below 10% which is unsatisfactory.

Figure 5 shows our 9 different budgets competing against various levels of Stockfish. We can notice that our best performing engines are depth-bound 4 and 5, with depth-bound 4 doing better. This indicates that depth-bounding Stockfish harms it more than it harms us. This is probably because depth-bounding Stockfish also prevents it from using simple alternative searches as much because these searches usually go deep. The reasoning for time-bound searches being ineffective in our competitive analysis is likely because Stockfish has a lower effective branch factor than our engine (~ 2 compared to ~ 5) which means even if it is less parallelizable it can still likely go significantly deeper than our engine. Finally, cycle-bound has arguably the worst performance out of the three, despite us expecting a slight edge over Stockfish. Where we might have gone wrong with this expectation is our cycles per node. Stockfish outputs how many nodes it searched, but not what algorithm it used to search them. Thus, if the majority of nodes in our calibration were searched with simpler techniques than ABP it could skew the cycle-bound performance in favor of Stockfish.

4.4. MPI Performance

All of our above benchmarking and competitive game performance is using just OpenMP threads, and the reason for that is because while we tested MPI implementations, we found that they did not scale better than just increasing the number of OpenMP threads. Or if they did, they used a naive method to avoid too many MPI messages which in turn causes the work to be mostly ineffective. We can conclude that for chess search trees a shared memory model is more important than just more compute capability at the cost of a distributed memory model.

In the case of ABP, splitting root parallelism across multiple ranks would have been too costly due to synchronizations needed for the frequent pruning steps. Thus, we implemented Lazy SMP across the MPI ranks and root parallelism within a single rank. However, even with this simpler implementation you still need to share entries from the TT across ranks in order to get proper move ordering (biggest node pruning tool). This TT sharing entry is done asynchronously to be efficient, but even so it is not possible to share the whole TT like in the OpenMP case which makes it way more likely for unnecessary work to be done. In terms of performance a 2x32 setup using OpenMP only led to a 2.3× speedup, while a 2x32 setup using OpenMP and MPI led to a 1.33× speedup. We had originally hoped that double the cache space (a TT table per rank) might improve performance, but it turns out it does the opposite because TT entries affects move ordering.

In the case of MCTS, performing tree parallelism across multiple ranks would have again been too costly due to synchronizations needed to maintain a single tree state that is frequently updated. Thus we implemented the more embarrassingly parallel, but also more naive parallelism of Root Parallelism. Like mentioned earlier this type of parallelism requires many more simulations to be effective due to the different roots being independent and duplicating work. In terms of performance a 64 thread setup using OpenMP only led to a 4.1× speedup, while a 2x32 setup using OpenMP and MPI led to a 24.9× speedup across all game stages (mid game only had a 38.9× speedup). This speedup is consistent with the greater speedups that Root Parallelism achieves compared to Tree Parallelism, but it will likely not lead to improved competitive game performance.

5. Challenges and Obstacles

Initially, we planned to explore WSE-2 (CSL) and potentially even GPU (CUDA) for the chess search tree problem to utilize the highly parallel nature of rollouts/shared memory model of CUDA and the lower communication overheads on the WSE-2 platform. However, through implementing the project we realized that while search trees may appear highly parallel at first, they actually have a significant amount of irregular control flow, irregular workloads, and pointer-heavy tree structures that are poorly suited for SIMD-style accelerators and CSL development (for example, compared to matrix multiplication or other compute heavy operations). Beyond the theoretical aspect, the main physical challenge was a lack of chess or bitboard libraries for CUDA/CSL. Basic functionality such as legal move generation and bitboard move application would have required substantial low-level reimplementation, which would have consumed the majority of the project timeline and distracted from the development of parallel algorithms.

Across the implemented algorithms, the irregularity of search trees introduced major load-balancing difficulties. In ABP, some threads would encounter highly prunable branches and terminate quickly, while others explored large subtrees with minimal pruning. This created workload imbalance across threads performing root parallelism which wasn't fixable through dynamic scheduling because root children were limited to 1 or 2 per threads. Improving the absolute efficiency of ABP through better pruning heuristics often reduced PE because there was less useful work remaining to distribute to

justify parallel overhead and more pruning meant more irregular workloads. In the end, while we did get a faster speedup than [4] (18.8× vs 3.23×), it was at the cost of an entire Perlmutter node with horrible parallel efficiency.

We also encountered synchronization bottlenecks in TP MCTS. While RP scaled well due to thread independence, it wasted compute because each thread statistically converged to the same win distributions. TP reduced this redundancy by sharing information between threads, but introduced contention on statistics and expansion logic. This was addressed by implementing lock-free atomics, virtual losses, and thread-local memory pools; however, cache traffic and atomic overhead limited scaling at higher thread counts. This demonstrated a tradeoff between sample/algorithm efficiency and parallel scalability.

Although MCTS (in root parallelism and most of tree parallelism) exhibited stronger scaling properties than the equivalent ABP speedup it underperformed in actual chess games. Chess has a relatively low branching factor and highly tactical positions where overlooking a single capturing sequence can be catastrophic. ABP's complete search was more reliable in these situations, whereas MCTS spent large portions of its simulation budget exploring irrelevant continuations. This result was consistent with our original hypothesis that stochastic search is less naturally suited for chess. However, if you consider the situation where a complete search is impossible due to large branching factors like in Go then MCTS is likely to be one of the better search algorithms you can choose.

Overall, the project revealed that achieving strong search efficiency and parallel scalability are often conflicting objectives. Techniques that improve pruning quality or search focus frequently reduce the amount of parallel work available, while approaches that maximize scalability often duplicate computation or weaken search quality. This tradeoff became the central insight of the project and shaped nearly all of our design decisions. In terms of the chess search tree, strong search efficiency appears to be better than parallel scalability, and we think that likely holds true for other game search tree as well. However, this may not be the case for more computational intensive (at least in terms of FLOPs) problems.

References

[1] Disservin, *chess-library: A modern, header-only C++ chess library*, GitHub repository, 2024. `https://github.com/Disservin/chess-library`, Accessed: 2026-05-01.

[2] Karl Kurzer, Christoph Hörtnagl, and J. Marius Zöllner, *Parallelization of Monte Carlo Tree Search in Continuous Domains*, arXiv preprint arXiv:2003.13741, 2020. `https://arxiv.org/abs/2003.13741`

[3] Lichess.org, *Lichess Puzzles Database*, `https://database.lichess.org/#puzzles`, Accessed: 2026-05-01.

[4] Shubhendra Pal Singhal and M. Sridevi, *Comparative Study of Performance of Parallel Alpha Beta Pruning for Different Architectures*, 2019 IEEE 9th International Conference on Advanced Computing (IACC), pp. 115–119, 2019. `https://ieeexplore.ieee.org/document/8971591`

[5] Stockfish Developers, *Stockfish: A strong open-source chess engine*, GitHub repository, 2024. `https://github.com/official-stockfish/Stockfish`

[6] Wiki Contributors, *Chess Programming Wiki: A useful repository of chess programming techniques*, `https://www.chessprogramming.org/Main_Page`, Accessed: 2026-05-01